



Green University of Bangladesh

*Department of Computer Science and Engineering (CSE)
Semester: Summer, , Year: 2026), B.Sc. in CSE (Day)*

HealSync - Advanced Digital Healthcare Portal

*Course Title: Web Programming Lab
Course Code: CSE 302
Section: 242 D7*

Students Details

Name	ID
Anirban Bosu	242002096
Jannatus Samaniya Chowdhury	242002042

*Submission Date: 03/08/2026
Course Teacher's Name: Mr. Mahbubur Rahman*

[For teachers use only: **Don't write anything inside this box**]

<u>Lab Project Status</u>	
Marks:	Signature:
Comments:	Date:

Contents

1	Introduction	2
1.1	Overview	2
1.2	Problem Definition	2
1.2.1	Problem Statement	2
1.2.2	Complex Engineering Problem	3
1.3	Design Goals/Objectives	3
1.4	Application	5

Chapter 1

Introduction

1.1 Overview

The primary objective of HealSync is to establish a seamless, zero-refresh user experience by implementing an asynchronous, tab-based navigation system that handles user registration, scheduling, and billing concurrently on a single web page. To ensure data retention across different browser sessions without a traditional server, the project integrates the browser-native LocalStorage API to cache and persist data states locally. Furthermore, the application achieves independent client-side image processing by utilizing the JavaScript FileReader API, which successfully encodes user-uploaded profile pictures into compact Base64 strings for direct localized storage. Finally, to eliminate public security vulnerabilities, a custom triple-click event listener is programmed onto the main application logo, triggering a hidden security pin modal that protects the administrative gateway from public visibility.

1.2 Problem Definition

1.2.1 Problem Statement

Traditional digital healthcare platforms and hospital management interfaces suffer from heavily fragmented and counter-intuitive user experiences. Arriving patients frequently encounter severe friction, requiring them to navigate a maze of disconnected pages, endure continuous browser reloads, and tolerate high latency just to complete basic clerical tasks such as registering a new profile, specifying critical medical data like blood groups, or processing emergency digital billing. This navigational fragmentation not only causes significant operational delays at clinical reception desks but also leads to higher rates of user drop-off and critical data entry errors during high-stress medical situations. Furthermore, conventional systems rely heavily on persistent backend server connectivity and external cloud infrastructure to host and render patient media, which induces performance lag and inflates setup costs in localized environments or temporary medical camps.

Most critically, standard web-based medical portals expose their administrative and

staff authentication gateways prominently within public navigation menus or highly predictable URL sub-directories. This structural design flaw presents a major security vulnerability by providing malicious actors with an exposed attack surface, making the core administrative system highly susceptible to automated brute-force attacks, credential harvesting, and unauthorized security breaches. Consequently, there is an urgent engineering need for a lightweight, serverless digital healthcare application that streamlines patient workflows into a unified, zero-refresh interface while simultaneously embedding a hidden administrative layout to eliminate public-facing security targets entirely.

1.2.2 Complex Engineering Problem

The core engineering challenge of HealSync lies in managing a relational, data-driven system entirely within a stateless, serverless client-side framework. Lacking a traditional database backend, the system must simulate complex relational mappings—such as binding unique patient IDs to specific billing records and appointments—inside the flat, synchronous key-value schema of the browser’s native `LocalStorage`. This architectural constraint requires optimized JavaScript serialization and array manipulation routines to maintain data integrity and prevent blocking the browser’s main execution thread during heavy data processing.

Another critical technical bottleneck involves processing and rendering multimedia assets without a backend file server or cloud storage infrastructure. HealSync intercepts raw binary image files from user inputs and converts them into monolithic Base64-encoded ASCII strings using the asynchronous `FileReader` API. Because Base64 encoding inflates original file sizes by roughly 33

Finally, enforcing access control and state management on an open, client-inspectable codebase presents a severe logical paradox. To isolate the administrative control panel without server-side authentication, the system relies on vanilla JavaScript routines to handle direct DOM mutations and manage a completely hidden gateway. This requires a precise, time-delta thresholding algorithm to differentiate an intentional administrative triple-click sequence from accidental user double-clicks, ensuring that the administrative layout and analytical calculation modules are conditionally injected without leaking sensitive structures to the public DOM tree.

1.3 Design Goals/Objectives

3. Design Goals and Objectives

3. Design Goals and Objectives

- **Serverless Data Management:** Build a fully client-side platform managing relational patient and financial datasets within the browser’s native `LocalStorage`.
- **Client-Side Media Processing:** Utilize the JavaScript `FileReader` API to convert binary images into Base64 strings, optimizing structures to stay

Table 1.1: Summary of the attributes touched by the mentioned projects

Name of the P Attributes	Explain how to address
P1: Depth of knowledge required	Deep mastery of vanilla JavaScript for structural client-side relational storage, asynchronous media processing, and main-thread optimization. Advanced proficiency in time-delta gesture security algorithms and zero-framework responsive UI design (Bento Grid/Glassmorphism).
P2: Range of conflicting requirements	Balancing heavy Base64 media storage within a strict 5MB browser limit while enforcing an invisible administrative access layer on an open, client-inspectable codebase.
P3: Depth of analysis required	Quantifying client-side thread latency and Base64 storage inflation while mathematically optimizing click-interval time deltas to eliminate security false-positives.
P4: Interdependence	The system relies on absolute hardware-software synchronization, where any operational failure between ultrasonic distance sensing, Arduino microprocessing, and servo motor actuation completely invalidates the dustbin's automated functionality.

within the 5-MB storage quota.

- **Optimized Single-Page UI:** Deliver a fluid, zero-refresh user interface using Bento Grid and Glassmorphic aesthetics without causing layout latency or thread-blocking.
- **Obscurity-Based Security:** Secure the administrative gateway via a custom time-delta gesture recognition algorithm (triple-click), completely isolating sensitive code structures from the public DOM.

1.4 Application

The HuffSecure Zip System can be used in file compression and secure data storage where both efficiency and security are required. It is applicable in reducing large file sizes using Huffman Coding while ensuring data protection through password-based encryption. This system can be used in secure file sharing, data backup, cloud storage systems, and educational purposes for understanding compression and cryptography. It is also useful in developing custom file archiving tools that combine both compression and security features in a single system.